TEMORA

Authentication & Identity Architecture

Implementation guide, request flows, data model, background processing, and operational review

Repository: github.com/huzaifaikhlaq/Temora

Branch examined: main

Review date: November 9, 2026

Scope: Next.js authentication BFF, Django Identity Service, PostgreSQL, Redis, Celery, Docker Compose.

Audience

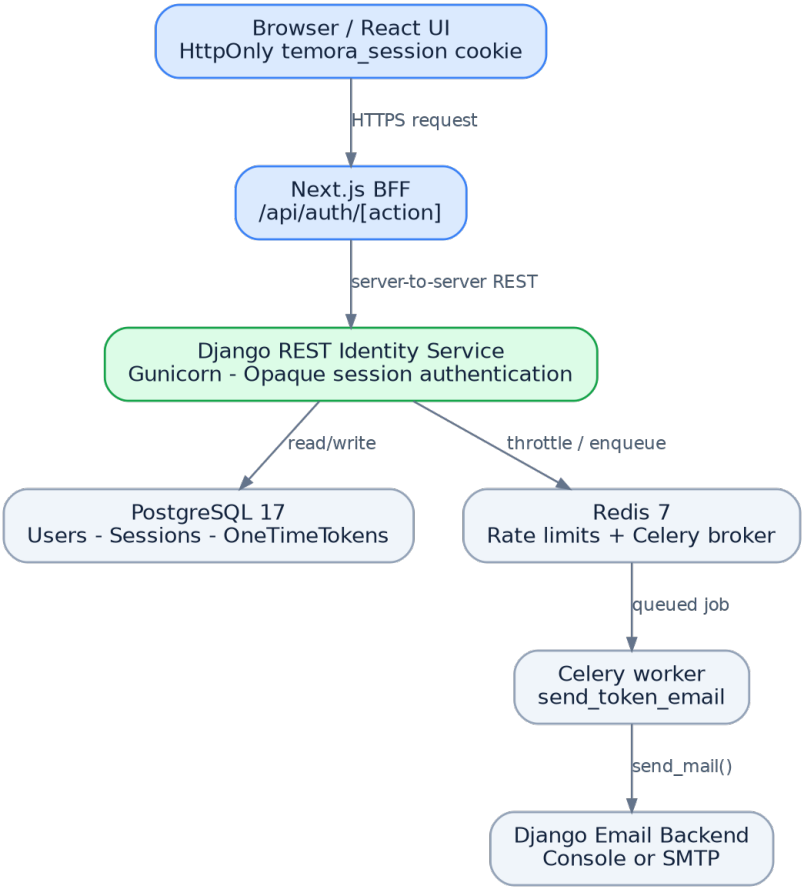
Developers onboarding to Temora, reviewers, backend maintainers, and engineers integrating future microservices.

Evidence note

This document describes inspected repository code. It does not claim production deployment, successful email delivery, full end-to-end runtime tests, or integration of unfinished services.

1. Executive architecture summary

Temora separates the browser-facing application from identity verification and session state. The Next.js backend-for-frontend (BFF) forwards authentication calls to a standalone Django REST API.



Critical design choice: Login uses a random opaque bearer token, not JWT. The raw token is placed into an HttpOnly browser cookie by Next.js. Only its SHA-256 digest is retained in the PostgreSQL session record.

Authentication verifies who a user is. Workspace-specific authorization (membership and role inside each tenant) must be enforced separately by domain services.

This repository also includes several domain-service folders. The inspection did not establish completed cross-service integration or a functioning gateway implementation.

2. Technology stack & boundaries

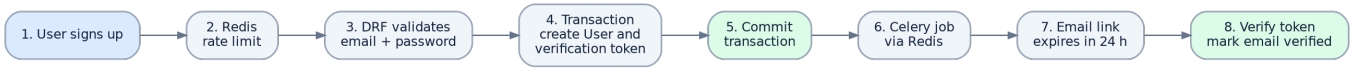
Component	Technology	Purpose
Browser UI	React / Next.js / TypeScript	Sign-in, sign-up and dashboard pages
BFF	Next.js route handlers + server-only fetch	Cookie handling; proxy to identity API; safe JSON response
Identity API	Python 3.12 / Django / DRF / Gunicorn	Credentials, token issuance, validation and revocation
Database	PostgreSQL 17 via psycopg	Users, persistent sessions and one-time tokens
Password hashing	Argon2 (PBKDF2 fallback)	Hash and verify stored passwords
Token generation	Python secrets + SHA-256	Cryptographic tokens; digests at rest
Rate limiting	Redis 7 + atomic Lua INCR/EXPIRE	Per-IP and optional email-key request limits
Async worker	Celery 5 + Redis broker	Verification and password-reset email delivery
Container runtime	Docker Compose	PostgreSQL, Redis, identity API, Celery worker

Responsibilities

- Browser: submits credentials and automatically sends the cookie; client JavaScript does not receive the raw session token in the login JSON.
- Next.js BFF: checks origin on POST, calls Django, sets/deletes **temora_session**, and forwards a bearer token for protected calls.
- Django: authenticates credentials, protects endpoints, queries session state, and coordinates email jobs.
- PostgreSQL: source of truth for identities, hashed sessions, and verification/reset tokens.
- Redis: rate-limit counters and Celery queue; not the authoritative session store.

3. Registration & email verification

Registration is not equivalent to an authenticated session. New users must prove ownership of their email before they can log in.



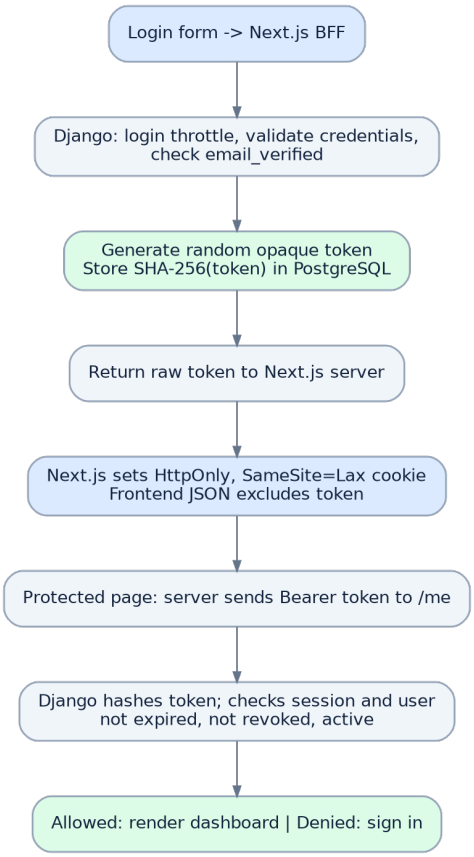
Step-by-step

- Next.js receives POST /api/auth/register and forwards the JSON to Django.
- The Django endpoint performs Redis rate limiting and DRF serializer validation, including the password policy.
- Inside transaction.atomic(), Django creates a user with email_verified=False and creates a one-time verification token.
- The database stores the SHA-256 token hash, while the email link contains the original token; verification links expire in 24 hours.
- send_token_email.delay_on_commit(...) submits work only after a successful transaction commit.
- The verification endpoint locks the matching unused token record, sets email_verified=True, and marks the token used.

The registration endpoint returns a generic 202 message even if an email is already registered, reducing account-enumeration exposure. Verification does not log the user in automatically.

4. Login, protected routes & session validation

A successful login creates a revocable database-backed session. A protected page does not merely trust the cookie: the Next.js server asks Django to validate the associated session.

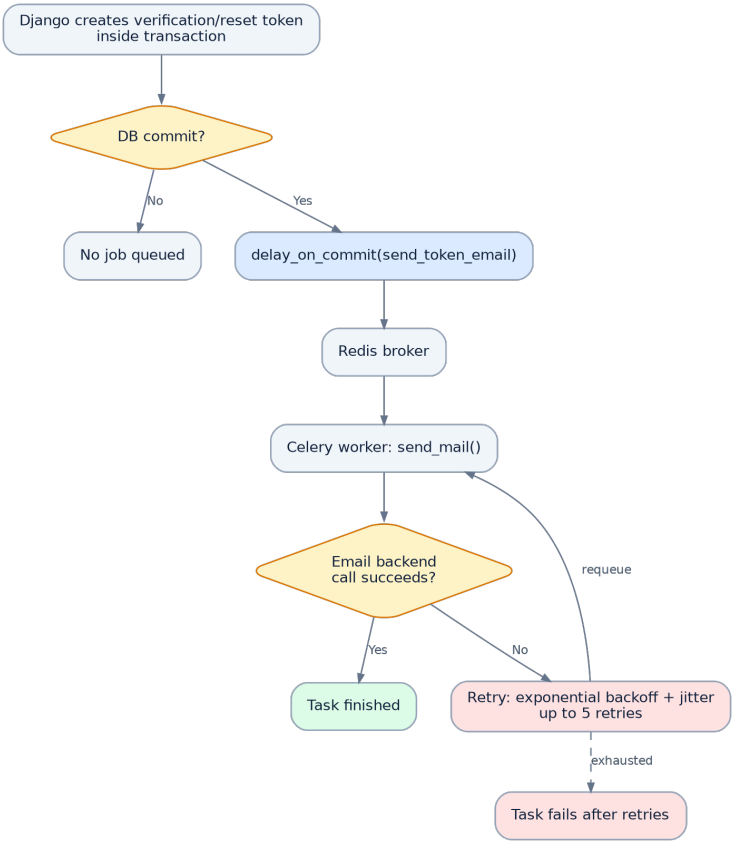


Token details

Property	Implementation
Token type	Random opaque string generated with secrets.token_urlsafe(48)
Lifetime	7 days (604800 seconds)
Stored in database	SHA-256 digest in Session.token_hash
Stored in browser	temora_session; HttpOnly; SameSite=Lax; Secure in production
Protected access	Next.js GET /api/auth/me -> Django bearer authentication
Validity conditions	Session exists; not revoked; not expired; user active

5. Background jobs: Redis & Celery

The application process enqueues email work instead of performing SMTP delivery during the registration or password-recovery HTTP request.



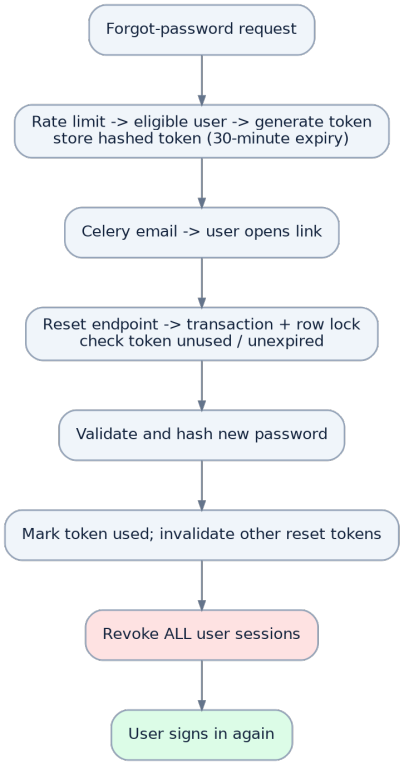
Existing task

accounts.tasks.send_token_email(email, purpose, url) selects either verification or reset-password content and calls Django send_mail().

Configured task behavior

- The worker runs celery -A config worker --loglevel=info --concurrency=2.
- Celery accepts JSON messages and uses Redis as broker. Task results are ignored; no result backend is configured.
- Errors trigger automatic retries with exponential backoff and jitter, up to 5 retries.
- The default email backend is console. Real delivery requires correct SMTP/provider settings.
- delay_on_commit prevents tasks being published for rolled-back transactions, but does not guarantee delivery if the process fails after commit but before publication.

6. Password recovery & logout



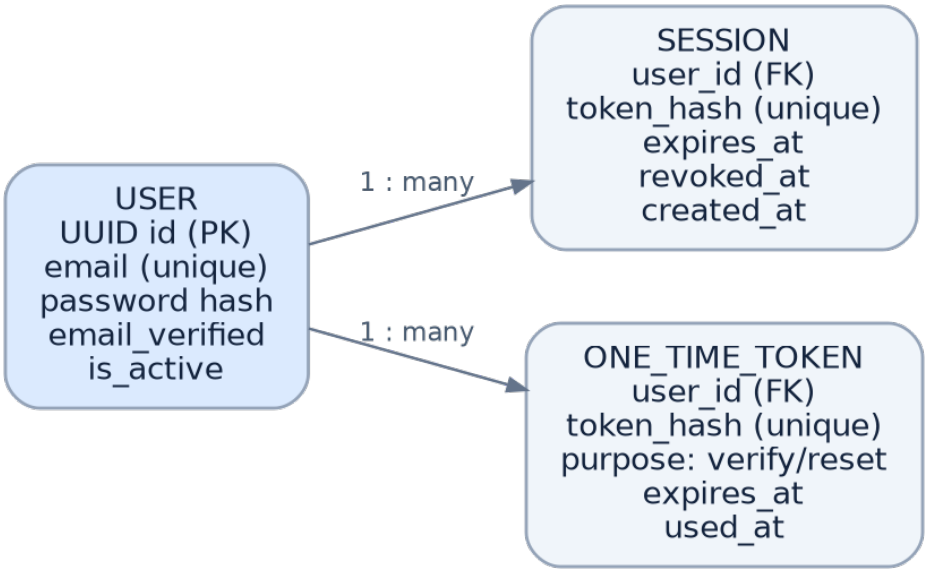
Reset flow

- Forgot-password uses a generic success message even when the account does not exist.
- The reset link expires after 30 minutes. The original token is not stored in plaintext.
- Under a database transaction, Django locks and validates the unused token, validates the new password, updates its hash, marks the token used, and invalidates outstanding reset tokens.
- Django revokes all sessions for that user after a successful password reset. Next.js deletes its local session cookie on a successful reset.

Logout flow

A valid logout calls POST /api/auth/logout. Django sets the current session's revoked_at timestamp. The Next.js route also clears the browser cookie. Note: the BFF presently clears the cookie even when the remote revocation request errors, so a token copy may remain valid until the server revokes or expires it.

7. Identity data model



Model semantics

- User: one global person identity, identified by UUID and unique email; includes password hash, active and verified flags.
- Session: one revocable login record. Multiple sessions can belong to one user, enabling multiple devices and per-session logout.
- OneTimeToken: short-lived, purpose-specific proof for email verification or password reset; consumed only once.

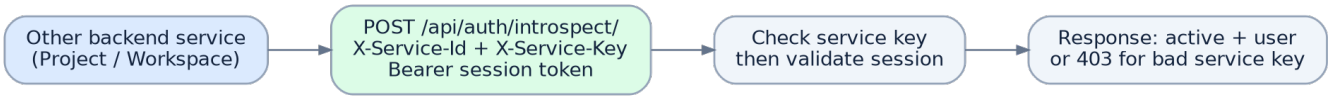
Why hashes match

At issue time: raw token -> SHA-256 -> stored hash. At verification time: received raw token -> SHA-256 -> database lookup. The same input deterministically yields the same digest. The application never needs to recover the original token from its hash.

PostgreSQL is the source of truth for active session state. Redis is not needed to find an already-issued login session, but the application currently depends on Redis for login rate limiting.

8. Service-to-service identity validation

Temora exposes a protected internal introspection endpoint intended for other domain services needing to validate a user session.



Endpoint contract

- POST /api/auth/introspect/ expects X-Service-Id and X-Service-Key, checked against configured service secrets using a timing-safe comparison.
- It also evaluates the supplied bearer session via OpaqueSessionAuthentication.
- An authorized caller receives active: true/false and a serialized user when authenticated. Incorrect service credentials result in 403.
- The endpoint itself does not grant workspace permissions, project membership, or tenant-scoped resource access.

Boundary / integration status

An introspection endpoint exists in Django, but this review did not confirm that Project, Workspace, Issue, Billing or Notification services call it. The gateway directory contains only .gitkeep; a dedicated gateway is not implemented in the inspected files.

9. Rate limits, operating environment & security

Endpoint group	Limit	Window
Register	5	1 hour
Login	10	60 seconds
Forgot / reset password	5	1 hour
Verify email	15	1 hour

Implementation

The Django throttle uses a Redis Lua script to atomically INCR a per-window counter and sets its expiration on first use. Keys are scoped by endpoint and hashed IP and (if present) email. The window is a fixed time bucket. A Redis failure returns HTTP 503, rather than silently disabling the throttle.

Operational configuration

- Docker Compose starts postgres:17, redis:7-alpine, Django identity API and Celery worker.
- Django uses Gunicorn with two workers in its Dockerfile and listens on port 8000 inside its container.
- Environment settings include POSTGRES_*, REDIS_URL, CELERY_BROKER_URL, FRONTEND_URL, DJANGO_SECRET_KEY and INTERNAL_SERVICE_KEYS_JSON.
- Default DRF permission is IsAuthenticated; public auth views explicitly override it.
- Next.js uses a POST origin check, strips token-like response fields and sets no-store for authentication responses.

10. Implementation review & follow-up work

These are improvement priorities, not claims that the current implementation is unusable.

Priority	Observation	Recommendation
High	Route handler exports runtime and dynamic while Next.js has cacheComponents enabled.	Remove incompatible segment config exports and confirm builds.
High	Remote logout can fail even as Next.js deletes its cookie.	Define retry/revocation behavior and make failure observable.
High	No confirmed automated end-to-end identity test suite in inspected sources.	Test registration, verification, login, expiry, logout, reset and worker retries.
Medium	Celery publishes after commit but without a durable outbox.	Consider transactional outbox when delivery guarantees matter.
Medium	Throttle keys use REMOTE_ADDR; proxy deployment may change the effective IP.	Establish trusted proxy/client-IP handling and test bypass resistance.
Medium	2FA UI is present but equivalent backend MFA flow was not found.	Treat MFA as incomplete until verified server-side.
Medium	Inter-service role enforcement is not established.	Implement tenant membership and permission checks in domain services.

Recommended acceptance test

Run: register -> confirm queued verification email -> verify -> login -> /me -> protected dashboard -> logout, then separately test forgotten-password reset and confirm every prior session fails authentication. Test Redis outages and email provider failures as well.

11. Source map & onboarding reference

Source path	What to read
src/lib/identity.ts	Server-only identity API client; cookies; origin helper
src/app/api/auth/[action]/route.ts	Next.js auth GET/POST proxy and cookie lifecycle
src/app/(dashboard)/layout.tsx	Server-side protected dashboard check
services/identity-service/accounts/views.py	Register, login, logout, me, verify, reset, introspect
services/identity-service/accounts/models.py	User, Session and OneTimeToken models
services/identity-service/accounts/services.py	Session issuance, token creation and Celery enqueue
services/identity-service/accounts/authentication.py	SHA-256 digest and opaque bearer verification
services/identity-service/accounts/throttles.py	Atomic Redis limits
services/identity-service/accounts/tasks.py	Email job implementation and retry policy
services/identity-service/config/settings.py	Database, Redis, DRF, Celery, email and security settings
services/identity-service/config/celery.py	Celery application and task discovery
compose.yaml	Database, cache, identity and worker services

Key terminology

BFF: backend-for-frontend; opaque token: a meaningless random secret whose validity is looked up server-side; introspection: a server-to-server check of a token; transaction commit: permanent database write of the transaction; message broker: queue that passes work to a worker; Celery worker: process that runs queued tasks.

Repository: https://github.com/huzaifaikhlaq/Temora

Document prepared from source review on November 9, 2026. Verify against newer commits before treating it as the operational specification.